Pontifícia Universidade Católica do Rio Grande do Sul
Escola Politécnica
Laboratório de Redes de Computadores

Interceptação e Defesa de um Protocolo de Aplicação

Relatório da Fase 2 — Interceptação Ativa (MITM)

Integrantes:
Rosana Schreiner Budant
Luísa Kirsch Silva Zarth
Leonardo Nunes Pasa

Porto Alegre — outubro de 2026

1. Objetivo

Este relatório documenta a Fase 2 do trabalho, cuja meta é demonstrar a interceptação ativa (man-in-the-middle). Um proxy posiciona-se entre o cliente e o servidor, repassa o tráfego nos dois sentidos e altera uma mensagem em trânsito, mostrando que, sem integridade nem autenticação do canal, o conteúdo pode ser adulterado sem que cliente ou servidor percebam. O proxy e o servidor atendem múltiplas conexões simultâneas, o que evidencia também a concorrência exigida pela fase.

2. Topologia e funcionamento da conexão

O redirecionamento é feito por configuração do próprio cliente: em vez de conectar diretamente no servidor (porta 9999), o cliente é apontado ao proxy (porta 8888) por meio da opção --porta 8888. O ponto central do ataque é que, com o proxy no meio, não existe mais uma única conexão ponta a ponta, e sim DUAS conexões TCP independentes, que o proxy mantém simultaneamente:

- Conexão 1 (cliente ↔ proxy): o cliente abre uma conexão TCP para a porta 8888. Do ponto de vista do cliente, essa porta é o "servidor", ele não sabe que fala com um intermediário. O proxy aceita essa conexão como um servidor faria.
- Conexão 2 (proxy ↔ servidor): assim que o cliente conecta, o proxy abre uma segunda conexão TCP, dele próprio para o servidor real na porta 9999. Nessa conexão, o proxy age como se fosse um cliente comum, o servidor não sabe que do outro lado está o proxy, e não o cliente original.

O proxy fica entre as duas conexões e faz o repasse (relay) do tráfego: tudo o que chega pela Conexão 1 (do cliente) é reenviado pela Conexão 2 (ao servidor), e tudo o que chega pela Conexão 2 (resposta do servidor) é reenviado pela Conexão 1 (ao cliente). No sentido cliente → servidor, antes de reencaminhar, o proxy inspeciona cada mensagem e, se for SET cor azul, a substitui por SET cor vermelho, as demais passam intactas.

```
cliente  --8888-->  proxy MITM  --9999-->  servidor
         <--------              <--------
```

Internamente, cada conexão de cliente é tratada pelo proxy em uma thread própria, e dentro dela há duas threads de repasse, uma para cada sentido do tráfego, já que cliente e servidor podem enviar dados ao mesmo tempo. Esse desenho permite ainda que o proxy atenda vários clientes simultaneamente, cada um com o seu par de conexões.

Por que cliente e servidor não percebem: para cada ponta, a sua conexão parece totalmente legítima. O cliente abriu uma conexão TCP e recebe respostas válidas; o servidor recebeu uma conexão e comandos bem-formados. Como o protocolo não possui verificação de integridade nem autenticação do canal, não há nada que denuncie a presença do intermediário. É exatamente essa ausência de proteção que o ataque explora, e que a Fase 3 corrige com TLS. Os três processos rodam no mesmo ambiente (GitHub Codespace), comunicando-se pela interface de loopback.

3. Demonstração da modificação

No modo de demonstração, o proxy intercepta toda mensagem SET cor azul e a troca por SET cor vermelho antes de encaminhá-la ao servidor. O cliente executou a sequência LOGIN, SET cor azul, GET cor e QUIT.

Lado do cliente: o cliente enviou “SET cor azul” e, ao consultar com GET cor, recebeu “OK vermelho”. Do ponto de vista dele, o valor que gravou foi silenciosamente trocado.

```
@RosanaBudant →/workspaces/lab-redes-intercept-defense/fase1-sniffer (fase2-proxy) $ python3 client.py --porta 8888
[*] Conectado em 127.0.0.1:8888
Comandos: LOGIN usuario senha | SET chave valor | GET chave | DEL chave | QUIT
> LOGIN alice senha123
[OK]  OK
> SET cor azul
[OK]  OK
> GET cor
[OK]  OK vermelho
> QUIT
[OK]  OK tchauzinho;)
[*] Conexão encerrada
```

Figura 1 — Cliente: envia SET cor azul e o GET cor devolve “vermelho”.

Lado do proxy: o proxy registra a mensagem original (INTERCEPTADO) e a versão adulterada (ALTERADO) que repassou ao servidor.

```
@RosanaBudant →/workspaces/lab-redes-intercept-defense/fase2-mitm (fase2-proxy) $ python3 proxy.py
[*] Proxy escutando em 0.0.0.0:8888
[*] Encaminhando para o servidor 127.0.0.1:9999
[*] Modo demonstracao: SET cor azul sera alterado para vermelho.
[*] Aguardando conexoes... (Ctrl+C para encerrar)
[+] Cliente conectado: ('127.0.0.1', 51160)
[+] Conectado ao servidor 127.0.0.1:9999
[*] Tunel aberto para ('127.0.0.1', 51160).
    [cliente -> servidor] b'LOGIN alice senha123\n'
    [servidor -> cliente] b'OK\n'
    [INTERCEPTADO] 'SET cor azul'
    [ALTERADO]     'SET cor vermelho'
    [servidor -> cliente] b'OK\n'
    [cliente -> servidor] b'GET cor\n'
    [servidor -> cliente] b'OK vermelho\n'
    [cliente -> servidor] b'QUIT\n'
    [servidor -> cliente] b'OK tchauzinho;)\n'
```

Figura 2 — Proxy: “[INTERCEPTADO] SET cor azul” e “[ALTERADO] SET cor vermelho”.

4. Evidência de rede (Wireshark)

Para comprovar a adulteração no nível da rede, o tráfego foi capturado com tcpdump nas portas 8888 e 9999 e aberto no Wireshark. Os dois pacotes abaixo mostram a mesma mensagem antes e depois de passar pelo proxy.

Antes do proxy (porta 8888): o pacote do cliente para o proxy (58150 → 8888) carrega “SET cor azul” (13 bytes de dados).

No.	Time	Source	Destination	Protocol	Lengtl	Info
15	0.003619	127.0.0.1	127.0.0.1	TCP	79	58150 → 8888 [PSH, ACK] Seq=22 Ack=4 Win=65536 Len=13 TSval=773972059 TSecr=773972059
16	0.003697	127.0.0.1	127.0.0.1	TCP	83	41182 → 9999 [PSH, ACK] Seq=22 Ack=4 Win=65536 Len=17 TSval=773972059 TSecr=773972059

```
> Frame 15: Packet, 79 bytes on wire (632 bits), 79 bytes captured (632 bits)
> Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00:00)
> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
> Transmission Control Protocol, Src Port: 58150, Dst Port: 8888, Seq: 22, Ack: 4, Len: 13
> Data (13 bytes)
```

```
0000  00 00 00 00 00 00 00 00  00 00 00 00 08 00 45 00   ·····E·
0010  00 41 10 96 40 00 40 06  2c 1f 7f 00 00 01 7f 00   ·A··@·@· ,·····
0020  00 01 e3 26 22 b8 0d 50  ab dd 0f 84 a7 b8 80 18   ···&"··P ·····
0030  02 00 fe 35 00 00 01 01  08 0a 2e 21 e0 5b 2e 21   ···5···· ···![·!
0040  e0 5b 53 45 54 20 63 6f  72 20 61 7a 75 6c 0a      ·[SET co r azul·
```

Figura 3 — Pacote cliente → proxy (porta 8888): conteúdo “SET cor azul”.

Depois do proxy (porta 9999): o pacote do proxy para o servidor (41182 → 9999) carrega "SET cor vermelho" (17 bytes). A diferença de tamanho (13 contra 17 bytes) confirma que o conteúdo foi alterado em trânsito.

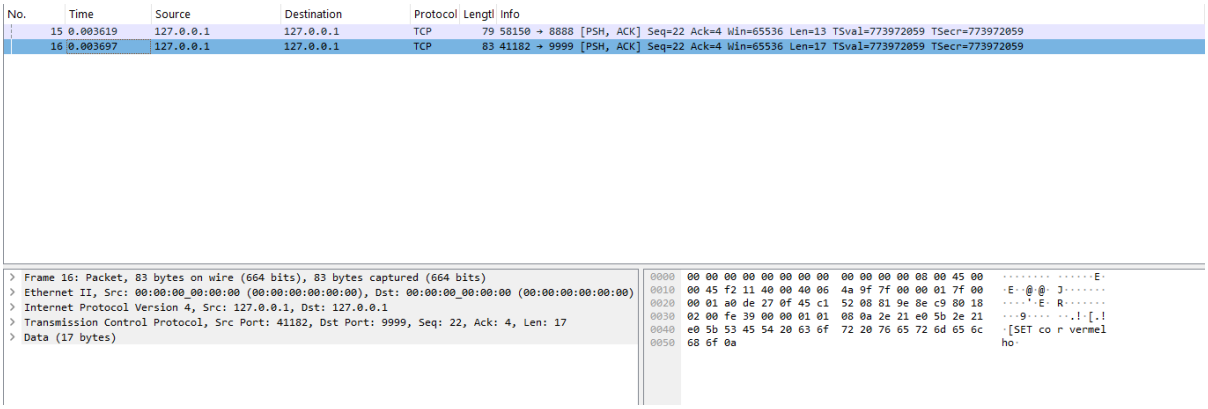


Figura 4 — Pacote proxy → servidor (porta 9999): conteúdo "SET cor vermelho".

A captura prova, de forma independente do log do proxy, que a mensagem entrou azul e saiu vermelha, ou seja, a quebra de integridade ocorreu e foi aceita como legítima pelo servidor. O sniffer passivo da Fase 1 não foi utilizado nesta fase porque o ataque aqui é ativo (exige alterar mensagens, e não apenas lê-las), o papel de evidência na rede é cumprido pela captura acima.

5. Concorrência (múltiplas conexões)

O proxy e o servidor atendem várias conexões ao mesmo tempo, usando o modelo de uma thread por conexão. Para comprovar, o script teste_carga.py dispara 20 clientes simultâneos, cada um com uma CHAVE e um VALOR distintos (cor0/valor0, cor1/valor1, ..., cor19/valor19), executando uma sessão completa (LOGIN, SET, GET, QUIT).

```
@RosanaBudant →/workspaces/lab-redes-intercept-defense/fase2-mitm (fase2-proxy) $ python3 teste_carga.py
[*] Disparando 20 clientes simultaneos contra 127.0.0.1:8888...

================================================
[*] Clientes disparados : 20
[*] Atendidos com sucesso: 20
[*] Falharam            : 0
[*] Tempo total         : 0.10 s
================================================
```

Figura 5 — Teste de carga (teste_carga.py): 20 clientes simultâneos, 20 atendidos, 0 falhas.

Todos os 20 clientes foram atendidos com sucesso e sem falhas. Como cada cliente usa uma chave própria, não há sobreposição de dados entre sessões, e o sucesso de todos confirma o atendimento simultâneo correto.

Evidência nos logs: a prova mais direta da simultaneidade está nos registros do servidor, onde as conexões aparecem intercaladas, o servidor processa um comando de uma conexão no meio do atendimento de outra. Se o atendimento fosse sequencial (um cliente por vez), as sessões apareceriam em blocos separados. Trecho do log do servidor durante o teste:

```
[RECV ('127.0.0.1', 36518)] SET cor2 valor2
[RECV ('127.0.0.1', 36562)] LOGIN alice senha123
[RECV ('127.0.0.1', 36520)] SET cor1 valor1
[RECV ('127.0.0.1', 36534)] SET cor0 valor0
[RECV ('127.0.0.1', 36518)] GET cor2
[RECV ('127.0.0.1', 36570)] SET cor6 valor6
```

Trecho do log do servidor: comandos de conexões diferentes (36518, 36562, 36520...) intercalados, comprovando atendimento simultâneo.

Segundo teste (alternativo): além do teste em Python, o projeto inclui teste_de_carga.sh, que sobe o servidor e o proxy automaticamente, dispara vários clientes em paralelo (processos em segundo plano) e grava a saída de cada um em arquivos de log. As duas abordagens são concorrentes e confirmam o atendimento simultâneo. O modelo de concorrência e as regiões críticas estão detalhados no documento CONCORRENCIA.md.

6. Conclusão

A Fase 2 atingiu seus objetivos: o proxy MITM intercepta e altera mensagens em trânsito, o servidor aceita o conteúdo adulterado como legítimo, e tanto a saída do proxy quanto a captura do Wireshark comprovam a quebra de integridade. O proxy e o servidor atendem múltiplas conexões simultâneas com robustez, sem que a falha ou o encerramento de uma sessão derrube as demais. Fica evidenciada a vulnerabilidade a adulteração e man-in-the-middle.